

PowerShell Command and Scripting Illustrated Guide

Commands • acronyms • script anatomy • OS compatibility • safe examples



WINDOWS

PowerShell 7 + Windows PowerShell 5.1

Use \$IsWindows / \$IsMacOS / \$IsLinux



macOS

PowerShell 7 (pwsh)

Use \$IsWindows / \$IsMacOS / \$IsLinux



LINUX

PowerShell 7 (pwsh)

Use \$IsWindows / \$IsMacOS / \$IsLinux

A practical field guide for beginners and working administrators. Examples favor full cmdlet names so scripts remain readable and portable. Compatibility labels describe the command itself; installed modules, permissions, and target services may add requirements.

1 How PowerShell works

PowerShell is a cross-platform command shell, scripting language, and automation platform. Its distinctive feature is an object pipeline: commands pass structured objects with properties and methods, rather than only plain text.



The pipeline passes rich .NET objects, so later commands can use properties—not just displayed text.

The Verb Noun naming pattern

Part	Meaning	Examples
Verb	The action. Approved verbs make commands predictable.	Get, Set, New, Remove, Test, Start, Stop
Noun	The resource or concept being acted on.	Process, Service, Item, Content, ChildItem
Parameter	A named input beginning with a hyphen.	-Path, -Name, -Filter, -Recurse
Argument	The value supplied to a parameter.	-Path './logs' → './logs' is the argument
Pipeline	Sends each output object to the next command.	Get-Process Sort-Object CPU

Read a command from left to right

```
Get-ChildItem -Path ./logs -File |
Where-Object Length -gt 1MB |
Sort-Object Length -Descending |
Select-Object -First 10 Name, Length
```

Translation: find files in ./logs, keep files larger than 1 MB, sort largest first, and show the first ten names and sizes.

2 Acronyms and terms

These are the abbreviations you will see most often in PowerShell documentation, scripts, administration, and automation.

Acronym	Expansion	What it means or does
PS	PowerShell	The shell and language. Also appears in names such as \$PSVersionTable.
CLI	Command Line Interface	A text-based interface used to run commands.
IDE	Integrated Development Environment	An editor with debugging and development tools, such as Visual Studio Code.
ISE	Integrated Scripting Environment	Legacy Windows-only PowerShell editor; available with Windows PowerShell, not the modern cross-platform default.
REPL	Read Evaluate Print Loop	The interactive prompt: type a command, run it, inspect the result.
Cmdlet	Command let	A small PowerShell command using Verb-Noun naming.
API	Application Programming Interface	A defined way for programs to communicate.
REST	Representational State Transfer	A common web API style used with Invoke-RestMethod.
JSON	JavaScript Object Notation	A portable text format for structured data.
CSV	Comma Separated Values	A row-and-column text format used for imports and exports.
XML	Extensible Markup Language	A tagged structured-data format.
WMI	Windows Management Instrumentation	Windows management infrastructure; older WMI cmdlets are Windows PowerShell oriented.
CIM	Common Information Model	A standards-based model used by Get-CimInstance for system management.
WinRM	Windows Remote Management	Windows implementation used by classic PowerShell remoting.
SSH	Secure Shell	Cross-platform secure remote access; PowerShell can use SSH transport.
DSC	Desired State Configuration	Declarative configuration technology for describing a desired machine state.
JEA	Just Enough Administration	Role-based constrained PowerShell endpoints on Windows.
ACL	Access Control List	Rules that define access to an object such as a file.
NTFS	New Technology File System	Windows file system commonly associated with Windows ACLs.
ENV	Environment	Process-level name/value settings exposed through the Env: drive.
URI / URL	Uniform Resource Identifier / Locator	An identifier or location for a web or network resource.
TLS	Transport Layer Security	Encryption used for network connections such as HTTPS.
LTS	Long Term Support	A release supported for a longer maintenance period.
CI CD	Continuous Integration and Continuous Delivery	Automated build, test, and deployment workflows.

3 Discovery and help commands

When you do not know a command, discover it from inside PowerShell. This habit is more dependable than memorizing aliases.

Command	What it does	Example	OS
Get-Command	Find available commands.	Get-Command -Verb Get -Noun Process	All
Get-Help	Read local help and examples.	Get-Help Get-Process -Examples	All
Update-Help	Download current help files; may require admin/network.	Update-Help	All
Get-Member	Inspect object type, properties, and methods.	Get-Process Get-Member	All
Get-Alias	Show aliases and their full commands.	Get-Alias ls	All*
Get-Module	List imported or available modules.	Get-Module -ListAvailable	All
Get-PSDrive	List PowerShell data stores/providers.	Get-PSDrive	All†
\$PSVersionTable	Display PowerShell and platform details.	\$PSVersionTable	All

* Alias sets can differ by platform. † Providers and drives differ—for example, the Registry provider is Windows-only.

Four questions to ask any object

```
$item = Get-Item ./example.txt
$item.GetType().FullName # What type is it?
$item | Get-Member # What can it expose or do?
$item | Format-List * # What values does it hold?
```

Aliases are nicknames, not separate commands. They are convenient interactively, but full cmdlet names are clearer in shared scripts. For example, `gci`, `dir`, and sometimes `ls` resolve to `Get-ChildItem`; exact aliases vary by platform and session.

Common syntax symbols

Symbol	Meaning	Example
\$	Variable	\$name = 'Ada'
@()	Array expression	\$ports = @(80, 443)
@{ }	Hashtable	\$user = @{ Name='Ada'; Role='Admin' }
\$_	Current pipeline object	Where-Object { \$_.CPU -gt 10 }
.	Property or method access	\$file.Length or \$text.ToUpper()
::	Static .NET member access	[Math]::Round(3.14)
#	Comment	# Explain why this step exists
`	Escape or line continuation; use sparingly	`n creates a newline in a double-quoted string

4 Core command sheet

File, data, and process commands are broadly portable in PowerShell 7. Provider-specific behavior and native applications can still differ.

Command	What it does	Example	OS
Get-ChildItem	List child items (files/folders).	Get-ChildItem -Path . -File -Recurse	All
Get-Item	Get one item and its object.	Get-Item ./report.csv	All
New-Item	Create a file, folder, registry key, etc.	New-Item ./archive -ItemType Directory	All*
Copy-Item	Copy items.	Copy-Item ./a.txt ./backup/	All
Move-Item	Move or rename items.	Move-Item ./old.txt ./new.txt	All
Remove-Item	Delete items; preview with -WhatIf.	Remove-Item ./temp -Recurse -WhatIf	All
Test-Path	Test whether a path exists.	Test-Path ./config.json	All
Get-Content	Read file content.	Get-Content ./app.log -Tail 50	All
Set-Content	Replace file content.	'ready' Set-Content ./state.txt	All
Add-Content	Append file content.	'next' Add-Content ./state.txt	All
Select-String	Search text and return match objects.	Select-String -Path *.log -Pattern 'error'	All
Import-Csv	Convert CSV rows into objects.	Import-Csv ./users.csv	All
Export-Csv	Write objects as CSV rows.	\$data Export-Csv ./out.csv -NoTypeInfo	All
ConvertFrom-Json	Convert JSON text to objects.	Get-Content ./a.json -Raw ConvertFrom-Json	All
ConvertTo-Json	Serialize objects as JSON.	\$data ConvertTo-Json -Depth 5	All
Get-Process	List processes.	Get-Process Sort-Object CPU -Descending	All
Stop-Process	Stop a process; requires appropriate rights.	Stop-Process -Id 1234 -WhatIf	All
Start-Process	Start a process; options differ by OS.	Start-Process 'https://example.com'	All*

5 Pipeline, output, and web commands

Command	What it does	Example	OS
Where-Object	Filter objects.	Get-Process Where-Object CPU -gt 10	All
Select-Object	Choose properties or limit results.	Get-Process Select-Object -First 5 Name,Id	All
Sort-Object	Sort objects by properties.	Get-ChildItem Sort-Object Length -Descending	All
Group-Object	Group items by a property/value.	Get-Process Group-Object ProcessName	All
Measure-Object	Count, sum, average, min, or max.	Get-ChildItem -File Measure-Object Length -Sum	All
ForEach-Object	Run a script block for pipeline items.	1..3 ForEach-Object { \$_ * 2 }	All
Format-Table	Format display as a table; use near the end.	Get-Process Format-Table Name,CPU	All
Format-List	Format display as a list.	Get-Item . Format-List *	All
Out-File	Send display output to a text file.	Get-Process Out-File ./processes.txt	All
Tee-Object	Save output and continue the pipeline.	Get-Process Tee-Object -FilePath ./p.txt	All
Invoke-RestMethod	Call a web API; parses structured responses.	Invoke-RestMethod https://api.example.com/items	All
Invoke-WebRequest	Request web content and response details.	Invoke-WebRequest https://example.com	All
Get-Random	Select random items/numbers.	Get-Random -Minimum 1 -Maximum 101	All
Get-Date	Get or format date/time.	Get-Date -Format 'yyyy-MM-dd HH:mm'	All
Read-Host	Prompt for interactive input.	\$name = Read-Host 'Name'	All
Write-Output	Write objects to the success stream.	Write-Output \$result	All
Write-Host	Write directly to the information/display host.	Write-Host 'Starting' -ForegroundColor Cyan	All*
Write-Warning	Write a warning record.	Write-Warning 'Low disk space'	All
Write-Error	Write an error record.	Write-Error 'Operation failed'	All

6 Which commands belong to which OS



WINDOWS

PowerShell 7 + Windows PowerShell 5.1

Use \$IsWindows / \$IsMacOS / \$IsLinux



macOS

PowerShell 7 (pwsh)

Use \$IsWindows / \$IsMacOS / \$IsLinux



LINUX

PowerShell 7 (pwsh)

Use \$IsWindows / \$IsMacOS / \$IsLinux

Cross-platform does not mean every module is cross-platform. The PowerShell language is highly portable; modules that manage Windows technologies often are not.

Area	Windows	macOS	Linux
Shell executable	pwsh (PowerShell 7); powershell.exe (Windows PowerShell 5.1)	pwsh	pwsh
Services	Get-Service works; Start-Service/Stop-Service support depends on service and permissions.	Get-Service visibility/support varies; use launchctl for native service management.	Get-Service support varies; systemctl is commonly used natively.
Registry	Registry provider and HKLM:/HKCU: drives.	No Windows Registry provider.	No Windows Registry provider.
Event logs	Get-WinEvent for Windows event logs.	Use native unified logging tools such as log show.	Use journalctl or distro log tools.
CIM / WMI	Get-CimInstance is commonly used; WMI technologies are Windows-centric.	Do not assume Windows CIM classes exist.	Do not assume Windows CIM classes exist.
Scheduled jobs	ScheduledTasks module, schtasks.exe, or task scheduler APIs.	Use launchd/launchctl.	Use cron/systemd timers.
Software packages	winget, Microsoft Store, MSI, package providers.	Homebrew or installer packages.	Distribution package manager such as apt, dnf, or pacman.
Permissions	Run elevated when required; ACLs often include NTFS concepts.	sudo and POSIX permissions; ACL tools differ.	sudo and POSIX permissions; ACL tools differ.

Portable OS branching

```
if ($IsWindows) {
  Get-WinEvent -LogName System -MaxEvents 20
} elseif ($IsMacOS) {
  & log show --last 1h
} elseif ($IsLinux) {
  & journalctl --since '1 hour ago'
}
```

The call operator & runs the native command. Test the native tool first with Get-Command log or Get-Command journalctl because availability and arguments depend on the host OS.

7 Script files and definitions

Extension / item	Definition	Use
.ps1	PowerShell script file	A runnable sequence of commands, parameters, functions, and control flow.
.psm1	PowerShell script module	Reusable functions and variables imported with Import-Module.
.psd1	PowerShell data file / module manifest	Stores configuration data or module metadata without ordinary imperative script flow.
.ps1xml	Formatting or type data file	Extends display/type information; used mainly by modules and advanced tooling.
Profile	A .ps1 loaded when a host starts	Customizes a user or host session; locate it with \$PROFILE.
Function	Named reusable script block	Packages logic, parameters, help, and output into a command.
Script block	Code enclosed in { }	Passed to cmdlets, invoked, or stored in a variable.
Module	Packaged reusable commands	Organizes functions/cmdlets and defines exported public commands.
Manifest	Metadata declaration	Describes module version, author, dependencies, compatible editions, and exports.
Transcript	Session text record	Created with Start-Transcript for auditing/troubleshooting where supported.

Anatomy of a safe portable script

```
#requires -Version 7.2
[CmdletBinding(SupportsShouldProcess)]
param(
    [Parameter(Mandatory)]
    [ValidateNotNullOrEmpty()]
    [string]$Path
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

try {
    if (-not (Test-Path -LiteralPath $Path)) {
        throw "Path not found: $Path"
    }
    Get-ChildItem -LiteralPath $Path -File
} catch {
    Write-Error "Scan failed: $($_.Exception.Message)"
    exit 1
}
```

Definition map

Construct	Purpose
#requires	Declares a required PowerShell version or module before execution.
[CmdletBinding()]	Gives a function/script advanced cmdlet behavior such as common parameters.

Construct	Purpose
param()	Defines named inputs and their types.
[Parameter()]	Adds parameter rules such as Mandatory or pipeline binding.
[Validate...]	Rejects invalid inputs early.
Set-StrictMode	Detects some uninitialized variables and unsafe syntax patterns.
\$ErrorActionPreference	Controls how non-terminating errors are treated; 'Stop' enables catch.
try / catch / finally	Handles errors and cleanup.
Write-Output / return	Emits objects; return also exits the current scope.
exit N	Ends the host process or script with an exit code; use carefully.

8 Scripting language quick reference

Need	Pattern	What it does
Variable	<code>\$name = 'Ada'</code>	Stores a value or object.
Array	<code>\$names = @('Ada','Lin')</code>	Stores an ordered collection.
Hashtable	<code>\$cfg = @{ Port=443; Secure=\$true }</code>	Stores key/value pairs.
If	<code>if (\$ok) { ... } elseif (...) { ... } else { ... }</code>	Chooses one branch.
Switch	<code>switch (\$state) { 'Ready' {...} default {...} }</code>	Matches one value against several cases.
Foreach loop	<code>foreach (\$item in \$items) { ... }</code>	Processes each item; clear and usually fast.
Pipeline loop	<code>\$items ForEach-Object { \$_ }</code>	Processes incoming pipeline objects.
For loop	<code>for (\$i=0; \$i -lt 10; \$i++) { ... }</code>	Repeats with an index/counter.
While	<code>while (\$condition) { ... }</code>	Repeats while a condition is true.
Function	<code>function Get-Thing { param([string]\$Name) ... }</code>	Defines a reusable command.
Comparison	<code>-eq -ne -gt -ge -lt -le -like -match -contains</code>	Compares values; operators are case-insensitive by default.
Boolean	<code>-and -or -xor -not / !</code>	Combines or negates conditions.
Null check	<code>if (\$null -eq \$value) { ... }</code>	Tests for no value; place \$null on the left.
Interpolation	<code>"Hello \$name"</code>	Expands variables in double-quoted strings.
Literal string	<code>'Hello \$name'</code>	Does not expand \$name.
Splatting	<code>Get-ChildItem @params</code>	Passes a hashtable of named parameters.
Custom object	<code>[pscustomobject]@{ Name='Ada'; Score=95 }</code>	Creates structured pipeline output.
Class	<code>class User { [string]\$Name }</code>	Defines a reusable custom type.

Function template

```
function Get-LargeFile {  
[CmdletBinding()]  
param(  
[Parameter(Mandatory, ValueFromPipeline)]  
[System.IO.FileInfo]$File,  
[long]$MinimumBytes = 1MB  
)  
process {  
if (-not $File.PSIsContainer -and $File.Length -ge $MinimumBytes) {  
$File # emit the original object  
}  
}  
}  
Get-ChildItem -File | Get-LargeFile -MinimumBytes 5MB
```

9 Running scripts safely

A script is code. Read it before running it, verify its source, understand its permissions, and test on non-production data. Use `-WhatIf` when a cmdlet supports it.

Task	Windows	macOS and Linux
Start PowerShell 7	<code>pwsh</code>	<code>pwsh</code>
Run a script in the current folder	<code>.\script.ps1</code>	<code>./script.ps1</code> or <code>pwsh ./script.ps1</code>
Run in the current scope	<code>. .\script.ps1</code> (dot sourcing)	<code>. ./script.ps1</code> (dot sourcing)
Pass arguments	<code>.\script.ps1 -Path C:\Logs</code>	<code>./script.ps1 -Path /var/log</code>
Check policy	<code>Get-ExecutionPolicy -List</code>	<code>Get-ExecutionPolicy</code> reports a value, but policy enforcement is a Windows feature
Make a Unix script directly executable	Not normally required for <code>.ps1</code> execution	Add <code>#!/usr/bin/env pwsh</code> , then <code>chmod +x ./script.ps1</code>
Find platform	<code>\$IsWindows; \$PSVersionTable</code>	<code>\$IsMacOS; \$IsLinux; \$PSVersionTable</code>

Execution policy is not a security boundary

On Windows, execution policy controls conditions for loading configuration files and running scripts. It is a safety feature and defense-in-depth measure—not malware protection or a permission system. Non-Windows systems do not enforce Windows execution policies in the same way. Avoid blanket recommendations to use `Bypass` or `Unrestricted`.

Safer change patterns

```
# Preview supported changes
Remove-Item ./staging -Recurse -WhatIf

# Confirm signature information on Windows
Get-AuthenticodeSignature ./Deploy.ps1

# Limit an execution-policy change to the current user on Windows
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

# Do not embed secrets; read them securely
$credential = Get-Credential
```

Common pitfalls

Pitfall	Better approach
Parsing formatted table text	Filter/select properties before <code>Format-Table</code> ; keep objects in the pipeline.
Using aliases in shared scripts	Use full names such as <code>Get-ChildItem</code> and <code>Where-Object</code> .
Assuming every error is caught	Use <code>-ErrorAction Stop</code> or set <code>\$ErrorActionPreference</code> intentionally.
Building paths with string concatenation	Use <code>Join-Path</code> and provider-aware commands.
Assuming Windows paths	Use <code>Join-Path</code> , <code>\$HOME</code> , and platform tests.
Deleting before validating	Resolve exact targets, test <code>Test-Path</code> , and use <code>-WhatIf</code> when available.

Pitfall	Better approach
Writing secrets to scripts or logs	Use secret stores, credentials, environment injection, or platform keychains.
Using Write-Host as data output	Emit objects; reserve Write-Host for intentional user-facing display.

10 Practical recipes

Find the ten largest files

```
Get-ChildItem -Path . -File -Recurse -ErrorAction SilentlyContinue |
Sort-Object Length -Descending |
Select-Object -First 10 FullName, Length
```

Build a portable system report

```
$report = [pscustomobject]@{
  Computer = [Environment]::MachineName
  OS = $PSVersionTable.OS
  Platform = $PSVersionTable.Platform
  PowerShell = $PSVersionTable.PSVersion.ToString()
  Timestamp = (Get-Date).ToString('o')
}
$report | ConvertTo-Json | Set-Content ./system-report.json
```

Call an API with error handling

```
try {
  $result = Invoke-RestMethod -Uri 'https://api.example.com/items' -TimeoutSec 20
  $result | Select-Object id, name
} catch {
  Write-Error "API request failed: $($_.Exception.Message)"
}
```

Import, transform, export

```
Import-Csv ./users.csv |
Where-Object Enabled -eq 'True' |
Select-Object Name, Email, Department |
Export-Csv ./enabled-users.csv -NoTypeInfo
```

Safe OS-aware configuration

```
$configRoot = if ($IsWindows) {
  Join-Path $env:APPDATA 'AcmeTool'
} else {
  Join-Path $HOME '.config/acmetool'
}
New-Item -Path $configRoot -ItemType Directory -Force | Out-Null
```

Debugging checklist

Question	Command or technique
What command will run?	Get-Command Name -All
What parameters exist?	Get-Help Name -Full
What object did I receive?	\$value.GetType().FullName; \$value Get-Member
What is the actual error?	\$Error[0] Format-List * -Force
Which line executes next?	Set-PSDebug -Trace 1 (disable with Set-PSDebug -Off)
Can I inspect without changing?	Use -WhatIf, -Verbose, Test-*, or a non-production target.
Which platform/version?	\$PSVersionTable; \$IsWindows; \$IsMacOS; \$IsLinux

11 Choosing the right tool

If you need to...	Start with...	Why
Inspect files or folders	Get-ChildItem, Get-Item, Get-Member	Returns structured file-system objects.
Search inside text files	Select-String	Returns match objects with file, line, and text.
Transform rows of data	Import-Csv → Where-Object → Select-Object → Export-Csv	Keeps the work object-based and auditable.
Read or write JSON	ConvertFrom-Json / ConvertTo-Json	Maps structured text to and from PowerShell objects.
Call a REST API	Invoke-RestMethod	Usually parses JSON/XML responses automatically.
Manage Windows-only infrastructure	The specific Windows module in PowerShell 7, or Windows PowerShell 5.1 if required	Module compatibility—not syntax—is usually the deciding factor.
Automate across OSes	PowerShell 7 plus portable cmdlets and explicit platform branches	Same language, with OS-specific edges isolated.
Create reusable commands	Advanced functions in a .psm1 module	Supports help, validation, pipeline input, tests, and versioning.
Schedule recurring work	Native OS scheduler calling pwsh -File script.ps1	Scheduling is OS-specific even when the script is portable.
Run commands on remote machines	PowerShell remoting over WSMan or SSH, depending on hosts	Transport, authentication, and endpoint configuration determine compatibility.

Compatibility decision path

Step	Check	Decision
1	Does the script require Windows-only modules, Registry drives, COM, WMI classes, or Windows event logs?	If yes, label it Windows-only or isolate those sections.
2	Does it call native commands such as ipconfig, launchctl, systemctl, sed, or grep?	Test with Get-Command and branch by platform.
3	Does it use path separators or fixed directories?	Replace concatenation with Join-Path and known folders.
4	Does it rely on PowerShell 7 language/features?	Add an appropriate #requires -Version line.
5	Does it need admin/root rights?	Detect prerequisites and fail with a clear message; do not silently elevate.

Step	Check	Decision
6	Does it modify data?	Add validation, backups where appropriate, -WhatIf support, and tests.

Recommended script header

```
# Purpose: one sentence describing the outcome
# Platforms: Windows / macOS / Linux (state tested versions)
# Requires: modules, tools, permissions, and PowerShell version
# Inputs: parameters, files, environment variables
# Outputs: object types, files, logs, and exit codes
# Safety: destructive effects, -WhatIf support, rollback
```

12 Reference notes and sources

Compatibility is a property of the complete environment: PowerShell edition and version, operating system, installed modules, native tools, permissions, architecture, and target service. Test important automation on every supported platform.

Authoritative references

- Microsoft Learn — What is PowerShell: <https://learn.microsoft.com/powershell/scripting/overview>
- Microsoft Learn — about Aliases:
https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_aliases
- Microsoft Learn — about Execution Policies:
https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies
- Microsoft Learn — Differences between Windows PowerShell 5.1 and PowerShell 7.x:
<https://learn.microsoft.com/powershell/scripting/whats-new/what-s-new-in-powershell-core-60>
- Microsoft Learn — About topics:
<https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about>

Legend

Label	Meaning
All	Available in PowerShell 7 on Windows, macOS, and Linux in ordinary installations.
All*	Broadly available, but aliases, parameters, host integration, or provider behavior may differ.
Windows	Requires Windows, a Windows-only module/provider/technology, or Windows PowerShell.
Native	Not a PowerShell cmdlet; availability and syntax come from the operating system or installed software.

Fast memory map

Intent	Verb family
Look without changing	Get, Test, Measure, Compare
Create	New, Add, Import
Change	Set, Update, Rename, Move, Convert
Remove or stop	Remove, Clear, Stop, Disable, Uninstall
Run or communicate	Start, Invoke, Send, Receive, Connect
Present or persist	Select, Sort, Group, Format, Export, Out

Use this sheet as a map, then verify the exact command with Get-Help and Get-Command on the machine where the script will run.